1. LITERATURE SURVEY

Accurate and robust localization is the fundamental prerequisite for the safe operation of Autonomous Vehicles (AVs). While traditional localization relies heavily on Global Navigation Satellite Systems (GNSS) and dead-reckoning, these methods suffer from signal degradation and unbounded cumulative drift, respectively. This literature survey systematically explores state-of-the-art localization techniques, focusing heavily on probabilistic filtering methods required to solve the "Kidnapped Vehicle Problem." In this extreme edge-case, a vehicle is initialized with a highly inaccurate global position and must deduce its true location using a map, noisy control data, and local sensor observations. We critically evaluate the transition from Extended Kalman Filters (EKF) to Particle Filters (Monte Carlo Localization), specifically detailing the mathematical formulations of the prediction, update, and resampling phases. Furthermore, this survey analyzes the computational complexity and real-time performance constraints associated with implementing 2D Particle Filters in C++ for edge-hardware deployment.

1.1 The Imperative of Autonomous Localization

The transition toward Level 4 and Level 5 autonomous driving relies on the vehicle's ability to precisely estimate its pose (position and orientation) within a globally referenced map [6]. Without reliable localization, downstream modules such as path planning, obstacle avoidance, and vehicle control cannot function, leading to catastrophic system failures [15].

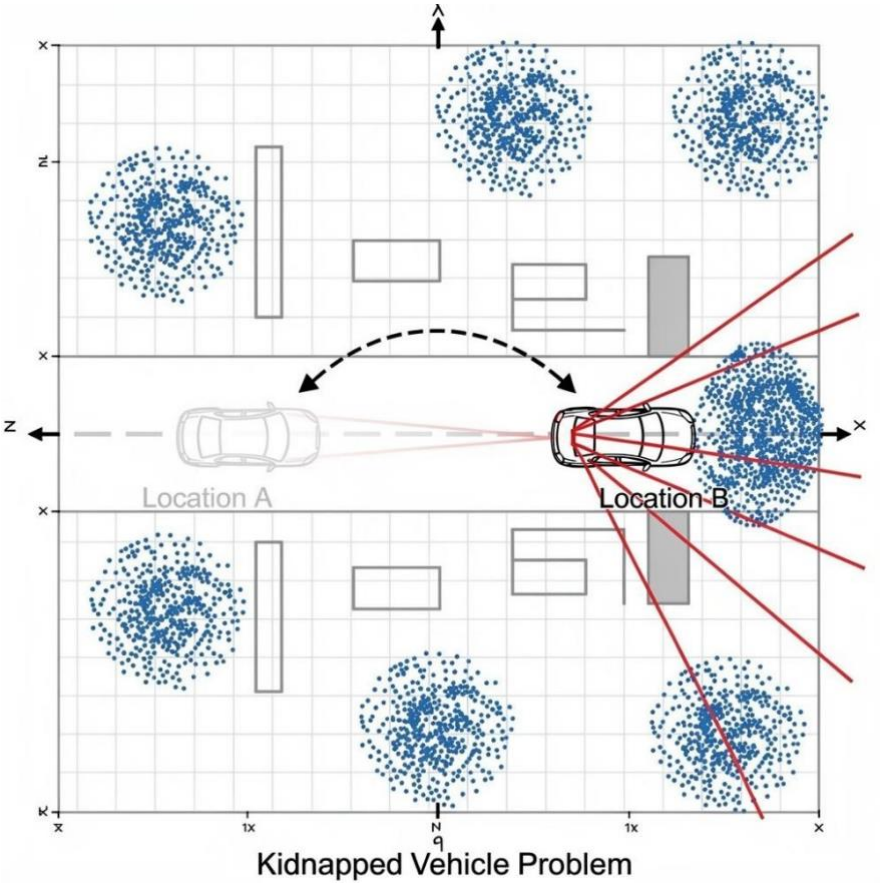
1.2 Defining the "Kidnapped Vehicle Problem"

The "Kidnapped Vehicle Problem" (KVP) is a classic and rigorous benchmark in mobile robotics. It simulates a scenario where a vehicle is instantaneously transported to a new, unknown location within a known map, or similarly, when a severe software fault causes the tracking algorithm to diverge entirely from reality [11].

In this scenario:

1. The vehicle possesses an accurate map of the environment.
2. The vehicle is given a highly noisy, potentially erroneous initial GPS estimate.
3. The vehicle receives continuous, but noisy, control data (velocity, yaw rate).
4. The vehicle receives continuous, noisy sensor observations (e.g., LiDAR distances to landmarks).

Solving the KVP requires a framework capable of maintaining multiple, distinctly separate hypotheses about the vehicle's state simultaneously, ruling out incorrect locations as new sensor data is acquired [14].



Kidnapped Vehicle Problem

2. Sensor Modalities and Noise Characterization

To construct an effective 2D Particle Filter, the system must accurately model the statistical noise profiles of the incoming data. Borenstein et al. [8] and Kuutti et al. [7] provide extensive classifications of sensor modalities.

2.1 Proprioceptive Sensors and Dead-Reckoning Drift

Proprioceptive sensors, such as wheel encoders and Inertial Measurement Units (IMUs), provide the vehicle with control data. However, integrating this data over time results in "dead-reckoning." Borenstein et al. [8] classify the resulting cumulative errors into two distinct categories:

- **Systematic Errors:** Caused by mechanical imperfections, such as unequal wheel diameters, misalignment of wheels, and uncertainty in the effective wheelbase.
- **Non-Systematic Errors:** Caused by dynamic environmental factors, such as wheel slippage, uneven terrain, and varying payload distributions. Because of these errors, the Particle Filter must inject Gaussian noise into the kinematic prediction step to ensure the particles explore the full range of possible actual movements.

2.2 Exteroceptive Sensors: GNSS and LiDAR

- **GNSS/GPS:** Global Positioning Systems provide absolute positioning but suffer from multi-path fading, atmospheric interference, and Non-Line-of-Sight (NLOS) errors, especially in urban canyons [12]. Yeong et al. [15] note that standard GPS can yield errors upwards of 10 meters. Therefore, GPS is solely utilized to provide a bounding box for the initial particle distribution [14].
- **LiDAR / Vision:** Light Detection and Ranging (LiDAR) provides highly accurate distance measurements to local landmarks. Wan et al. [4] demonstrated that fusing LiDAR with prior maps yields centimeter-level accuracy. However, LiDAR measurements contain Gaussian noise relative to the sensor's physical limitations, which must be modeled during the weight-update phase of the filter [5].

Table 1: Comprehensive Analysis of Sensor Noise Profiles for AV Localization [4], [7], [8], [15]

Sensor Modality	Primary AV Function	Error Profile / Challenges	Integration into Particle Filter Architecture
GPS / GNSS	Global absolute initialization	Multi-path fading, high variance, NLOS occlusion.	Establishes the initial X, Y, Θ distributi on bounds.
Wheel Odometry	Kinematic velocity tracking	Unbounded cumulative dead-reckoning drift.	Drives the Prediction step; requires added Gaussian noise.
IMU (Gyroscopes)	Yaw rate / orientation	Bias drift over time, temperature sensitivity.	Fused with odometry for the Θ state transition model.
LiDAR	Range-to-landmark tracking	Reflection errors, finite beam resolution limits.	Drives the Update step; mapped via Likelihood Field Models.
Vision (Cameras)	Semantic landmark detection	Illumination variance, severe weather occlusion.	Auxiliary data association; semantic feature matching.

3. Mathematical Foundations of Probabilistic Filtering

Because sensor data is inherently imperfect, autonomous localization is framed as a probabilistic Markov process. The goal is to estimate the posterior probability density function (PDF) of the vehicle's state x_t given all previous observations $z_{1:t}$ and controls $u_{1:t}$.

3.1 The Failure of Kalman Filters in KVP

Traditional systems utilize the Kalman Filter (KF), Extended Kalman Filter (EKF), or Unscented Kalman Filter (UKF). Reina et al. [9] demonstrated that adaptive fuzzy-logic KFs can handle sudden maneuvers. However, all KF

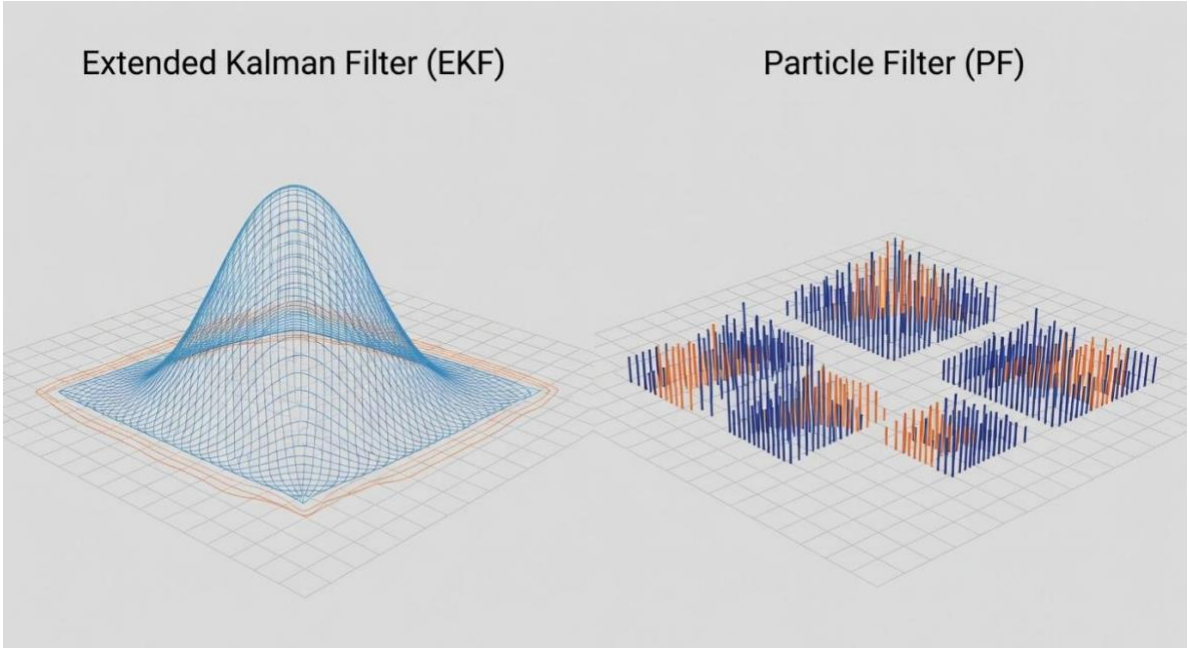
variants operate under the assumption that the probability distribution is strictly Gaussian (a single bell curve).

If a vehicle is kidnapped, the true probability distribution becomes highly multimodal (the vehicle could realistically be in Location A, B, or C simultaneously based on limited data). An EKF will forcefully average these hypotheses, placing the vehicle's estimated state in a completely empty space (e.g., inside a building), leading to irrecoverable divergence [11].

3.2 The Monte Carlo Approach

Monte Carlo Localization (MCL) relies on a Particle Filter. Instead of representing the PDF as a parameterized mathematical equation, the PF represents the PDF as a set of N discrete weighted samples (particles), defined as: $S_t = (x_t^{[1]}, w_t^{[1]}), (x_t^{[2]}, w_t^{[2]}), \dots, (x_t^{[N]}, w_t^{[N]})$

Where $x_t^{[i]}$ is the hypothesized state (pose) of the i^{th} particle, and $w_t^{[i]}$ is its corresponding importance weight [13]. This allows the filter to represent any arbitrary, non-Gaussian, multimodal distribution, making it the mathematically optimal solution for the Kidnapped Vehicle Problem [3].



4. Architecture of the 2D Particle Filter in C++

Implementing a 2D Particle Filter in C++ requires four distinct operational phases, executing in a continuous loop.

4.1 Phase 1: Initialization

The algorithm begins by generating a uniform or Gaussian distribution of particles around a noisy GPS estimate. According to de Miguel et al. [14], utilizing a GNSS bounding box drastically reduces computational overhead compared to scattering particles across the entire global map. If the initial GPS reading is $(x_{gps}, y_{gps}, \theta_{gps})$, the i^{th} particle is initialized using a normal distribution $N(\mu, \sigma^2)$:

$$x^{[i]} \sim N(x_{gps}, \sigma_x^2)$$

$$y^{[i]} \sim N(y_{gps}, \sigma_y^2)$$

$$\theta^{[i]} \sim N(\theta_{gps}, \sigma_\theta^2)$$

All initial weights are set to $w^{[i]} = \frac{1}{N}$

4.2 Phase 2: Kinematic Prediction

Upon receiving control inputs—velocity (v) and yaw rate (θ)—the state of each particle is projected forward in time Δt. To model the dead-reckoning drift documented by Borenstein et al. [8], Gaussian noise is injected into the control commands.

Using standard bicycle kinematic models, the state transition for each particle is:

If the yaw rate is non-zero ($\theta \neq 0$):

$$x_{t+1}^{[i]} = x_t^{[i]} + \frac{v}{\theta}[\sin(\theta_t^{[i]} + \theta\Delta t) - \sin(\theta_t^{[i]})] + N(0, \sigma_x^2)$$

$$y_{t+1}^{[i]} = y_t^{[i]} + \frac{v}{\theta}[\cos(\theta_t^{[i]}) - \cos(\theta_t^{[i]} + \theta\Delta t)] + N(0, \sigma_y^2)$$

$$\theta_{t+1}^{[i]} = \theta_t^{[i]} + \theta\Delta t + N(0, \sigma_\theta^2)$$

If the vehicle is moving straight ($\theta = 0$):

$$x_{t+1}^{[i]} = x_t^{[i]} + v\Delta t \cos(\theta_t^{[i]}) + N(0, \sigma_x^2)$$

$$y_{t+1}^{[i]} = y_t^{[i]} + v\Delta t \sin(\theta_t^{[i]}) + N(0, \sigma_y^2)$$

4.3 Phase 3: Data Association and Weight Update

This is the most computationally demanding phase of the C++ implementation. The vehicle uses LiDAR to measure the distances to local landmarks.First, the local sensor observations must be mathematically transformed into the global map coordinate space, assuming the particle's pose is the true vehicle pose [5]. This utilizes a 2D Homogeneous Transformation Matrix:

$$\begin{bmatrix} x_{map} \\ y_{map} \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & x_{part} \\ \sin\theta & \cos\theta & y_{part} \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_{obs} \\ y_{obs} \\ 1 \end{bmatrix}$$

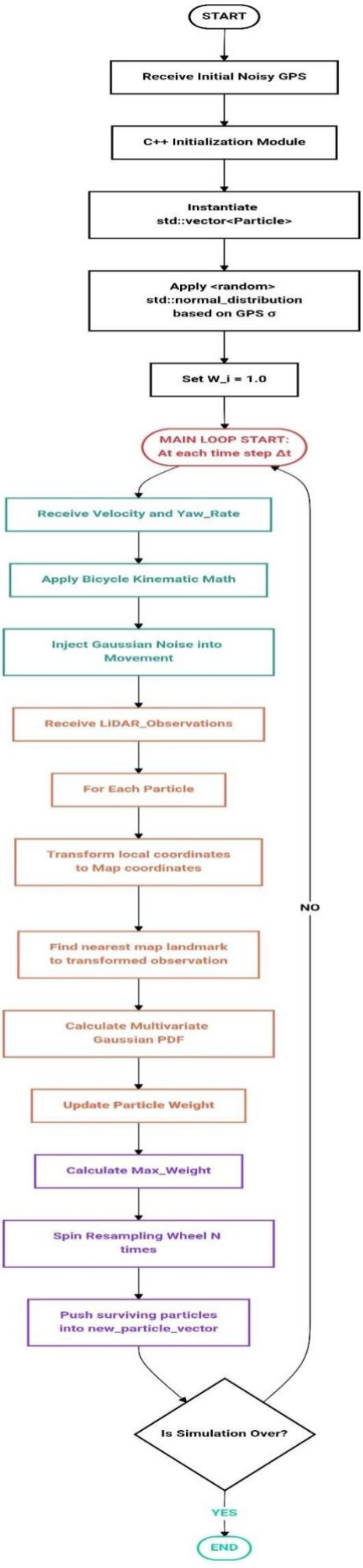
Once transformed, data association techniques (such as Nearest Neighbor) link the transformed observation to the closest actual landmark on the map. Finally, the weight of the particle is updated using a Multivariate Gaussian Probability Density Function. As detailed by Dhawale et al. [13], the weight reflects the likelihood of the sensor reading given the map:

$$w^{[i]} = \prod_{j=1}^{M} \frac{1}{2\pi\sigma_x\sigma_y} \exp\left(-\left[\frac{(x_{obs,j} - \mu_{x,j})^2}{2\sigma_x^2} + \frac{(y_{obs,j} - \mu_{y,j})^2}{2\sigma_y^2}\right]\right)$$

4.4 Phase 4: Resampling

The final phase applies the principle of "survival of the fittest." Particles with high weights are drawn multiple times to form the new particle cloud, while low-weight particles are destroyed. To ensure $O(N)$ computational efficiency in C++, the **Resampling Wheel** (Stochastic Universal Sampling) algorithm is commonly implemented, maintaining the overall particle count while refining the hypothesis density around the true location [13].

FLOWCHART 1:
C++ Particle Filter Architecture for the Kidnapped Vehicle



5: Advanced Solutions for Recovery (Adaptive MCL)

While standard MCL operates perfectly during normal driving, the pure "Kidnapped Vehicle Problem" requires adaptive logic. If the vehicle is kidnapped *after* the particles have already converged to a single location, all particles will suddenly register near-zero weights, causing the filter to crash.

5.1 Adaptive Monte Carlo Localization (AMCL)

De Miguel et al. [14] present AMCL as the industry standard for this issue. AMCL tracks the short-term average weight (W_{fast}) and long-term average weight (W_{slow}) of the particle cloud. If $W_{fast} < W_{slow}$, it indicates that the sensor data has suddenly stopped matching the map—the hallmark of a kidnapping event. In C++, this triggers a heuristic to inject completely random particles across the map, allowing the filter to "re-discover" its location.

5.2 GNSS Injection and Hierarchical Planning

To optimize AMCL, de Miguel et al. [14] proposed continuously using GNSS signals. Rather than injecting *purely* random particles during a kidnapping, the algorithm injects new particles based on the latest noisy GPS coordinate, significantly speeding up recovery time.

Taking a different approach, Zhou and Sakane [11] proposed a Hierarchical Bayesian Network. In their system, if the particle filter detects high variance (a kidnapping), a higher-level planner assumes control. It calculates the "Integrated Utility" of nearby regions and commands the vehicle to drive toward feature-rich areas (like intersections) to gather distinct LiDAR data, forcing the particle filter to converge faster [11].

Table 3: Strategies for Kidnapped Vehicle Recovery [1], [11], [14]

Recovery Methodology	Trigger Condition	Algorithmic Action	Computational Cost
Standard AMCL	$W_{fast} < W_{slow}$	Inject random particles globally.	High (Requires large N to cover map).

GNSS-Injected AMCL	High variance + GPS sync	Inject particles at noisy GPS bounds.	Medium (Bounded search space).
Hierarchical Planning	High particle variance	Vehicle executes deliberate search pattern.	Very High (Requires path planning).
Dynamic Exclusion	Conflicting landmark	Ignore observation (assume dynamic obstacle).	Low (Simple outlier rejection).

6. Computational Complexity and Real-Time C++ Performance

The transition from mathematical theory to a deployable autonomous system hinges entirely on computational complexity. Lu et al. [10] conducted a comprehensive review of real-time performance in vehicle localization, proving that high-level interpreted languages (like Python/MATLAB) cannot meet the latency constraints of autonomous driving (requiring sub-100ms updates).

6.1 Time Complexity Analysis

The Particle Filter's computational load is dictated by the number of particles (N), the number of observations (O), and the number of map landmarks (L).As defined by Akai et al. [5], the brute-force data association step has a complexity of $O(N \times O \times L)$. If $N = 1000$, $O = 50$, and $L = 1000$, a single time step requires 50,000,000 Euclidean distance calculations.

6.2 C++ Optimizations for Edge Hardware

To deploy this on constrained edge hardware, rigorous C++ optimization is required:

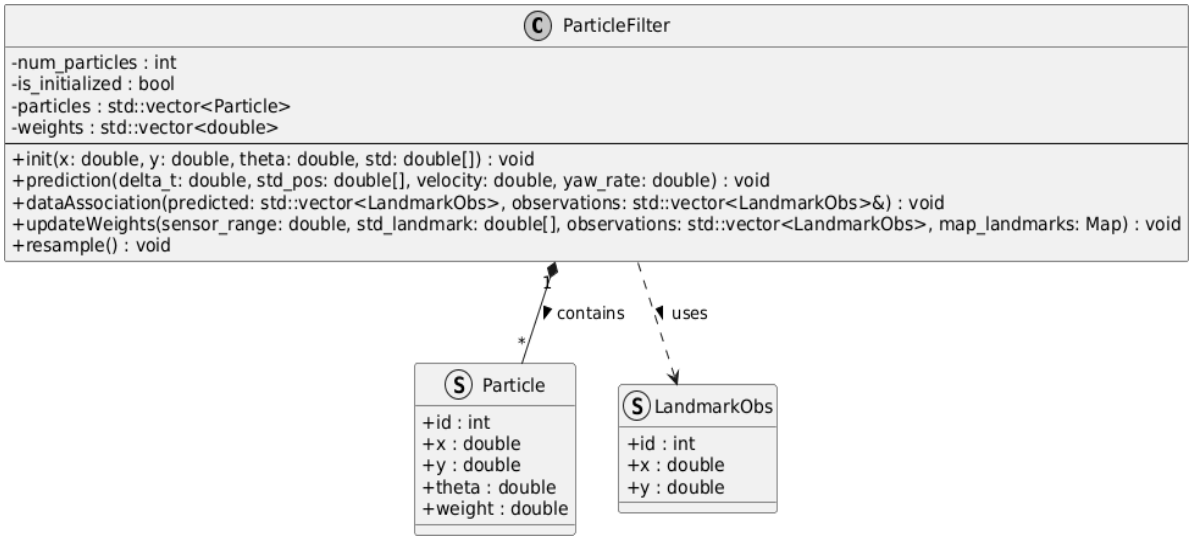
1. **Memory Contiguity:** Utilizing std::vector ensures contiguous memory allocation, drastically reducing cache misses in the CPU during the prediction and weight update loops.
2. **KD-Trees for Data Association:** To reduce the $O(O \times L)$ landmark search, the C++ implementation must pre-compute the map landmarks into a K-Dimensional Tree. This reduces the search complexity from $O(L)$ to $O(\log L)$, allowing the system to scale to massive city maps [10].

3. **Distance Field (DF) Representation:** As proposed by Akai et al. [5], instead of searching for landmarks, the map can be pre-rendered as a grid where each cell contains the distance to the nearest obstacle. The C++ code then achieves $O(1)$ constant time lookup for data association.

Table 4: Computational Complexity and C++ Optimization Strategies [5], [10], [13]

Phase of MCL	Naive Time Complexity	C++ Optimization Strategy	Optimized Complexity
Prediction	$O(N)$	Pass-by-reference updates, auto-vectorization.	$O(N)$ (High Speed)
Data Association	$O(N \times O \times L)$	KD-Trees / Distance Field Maps (DF).	$O(N \times O\log\ L)$
Weight Update	$O(N \times O)$	Pre-compute Gaussian constants; <cmath>.	$O(N \times O)$
Resampling	$O(N\log\ N)$	Stochastic Universal Sampling (Resampling Wheel).	$O(N)$

FLOWCHART 2:



7. Integration into Broader Autonomous Systems

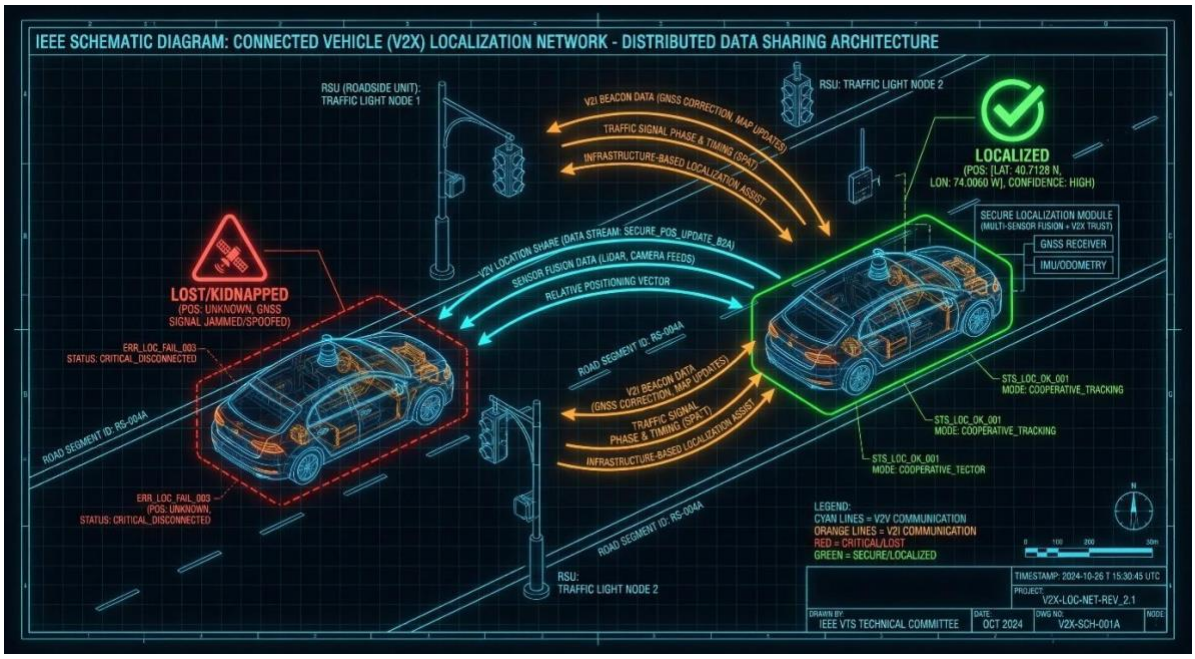
While the 2D Particle Filter solves the immediate Kidnapped Vehicle problem, it must interface with the broader Autonomous **Vehicle stack.**

7.1 Multi-Sensor Data Fusion

Wan et al. [4] emphasize that MCL should not exist in a vacuum. In diverse city scenes, their system fuses LiDAR MCL output with RTK-GNSS and IMU outputs using a final Error-State Kalman Filter. This ensures that even if the Particle Filter struggles with symmetry (e.g., a long straight tunnel), the IMU can bridge the gap. Kim and Lee [12] parallel this by fusing visual odometry into the mix, ensuring that lane-markings can provide lateral corrections to the MCL output.

7.2 Connected Vehicles (V2X) and Future Horizons

The ultimate evolution of vehicle localization relies on Vehicle-to-Everything (V2X) networks. Shen et al. [3] proved that by utilizing a Rao-Blackwellized Particle Filter across a network of vehicles, cars can share their pseudo-range observations. If Vehicle A is kidnapped, but Vehicle B (which is highly confident in its location) detects Vehicle A via LiDAR, Vehicle B can transmit a high-confidence bounding box to Vehicle A, allowing Vehicle A's Particle Filter to instantly collapse on the correct location. Furthermore, Faisal et al. [6] note that as city infrastructure evolves to support AVs, smart landmarks (such as RFID-enabled traffic lights) will fundamentally alter the data-association phase, providing 100% confidence landmark matching and eradicating the Kidnapped Vehicle Problem permanently.



8.Science Effects

The autonomous vehicle localization system is influenced by several scientific effects related to sensing, motion, and computation. Sensor noise plays a critical role, where GPS signals suffer from multi-path interference and non-line-of-sight errors, leading to inaccurate position estimates. Similarly, LiDAR and vision sensors are affected by environmental conditions such as lighting, weather, and object reflections.

Another important effect is dead-reckoning drift, where small errors from wheel encoders and IMU accumulate over time, causing large deviations in estimated position. Environmental effects like uneven terrain and wheel slippage introduce non-systematic errors, further degrading accuracy.

Additionally, computational effects influence system performance. High algorithmic complexity in particle filters can increase latency, making real-time localization challenging. Therefore, optimization techniques are necessary to balance accuracy and speed.

9.Concepts, Rules, Principles, Laws

9.1 Concepts

The key concept is autonomous localization, where a vehicle determines its position using sensors and maps. The Kidnapped Vehicle Problem describes a situation where the vehicle loses track of its location and must re-localize. This is addressed using Particle Filters (Monte Carlo Localization), which represent multiple possible positions simultaneously.

9.2 Rules

The system follows a cyclic process:

1. Initialization of particles
2. Prediction using motion data
3. Update using sensor observations
4. Resampling based on weights

Data association rules are also applied to match sensor observations with known map landmarks.

9.3 Principles

The system is based on probabilistic estimation, where uncertainty is modeled

using probability distributions. It follows the Bayesian principle, updating beliefs based on new sensor data. The resampling principle ensures that more probable states survive, similar to "survival of the fittest."

9.4 Laws

The system relies on mathematical laws such as:

- Gaussian distribution for noise modeling
- Markov assumption (current state depends only on previous state)
- Probability density functions for state estimation

10 Constants and Variables

10.1 Constants

Constants include fixed parameters such as sensor noise values (standard deviations), number of particles, and map landmarks. These remain unchanged during execution and define system behavior.

10.2 Variables

Variables include dynamic values such as vehicle position (x, y, θ), velocity, yaw rate, sensor observations, and particle weights. These change continuously as the vehicle moves and receives new data.

11 Equations & Relationships

1. Particle Initialization (Gaussian Distribution)

$$x^{[i]} \sim \mathcal{N}(x_{gps}, \sigma_x^2), y^{[i]} \sim \mathcal{N}(y_{gps}, \sigma_y^2), \theta^{[i]} \sim \mathcal{N}(\theta_{gps}, \sigma_\theta^2)$$

Relationship:

Each particle's initial position depends on the GPS estimate, but spreads based on noise (σ). Larger σ → wider uncertainty.

2. Motion (State Transition Model)

For turning motion:

$$x_{t+1} = x_t + \frac{v}{\dot{\theta}}[\sin(\theta_t + \dot{\theta}\Delta t) - \sin(\theta_t)]$$

$$y_{t+1} = y_t + \frac{v}{\dot{\theta}}[\cos(\theta_t) - \cos(\theta_t + \dot{\theta}\Delta t)]$$

$$\theta_{t+1} = \theta_t + \dot{\theta}\Delta t$$

For straight motion:

$$x_{t+1} = x_t + v\Delta t\cos(\theta_t), y_{t+1} = y_t + v\Delta t\sin(\theta_t)$$

Relationship:

Position (x, y) depends on velocity (v), direction (θ), and time (Δt). Yaw rate ($\dot{\theta}$) controls turning — if $\dot{\theta} = 0$ → straight motion.

4. Weight Update (Multivariate Gaussian)

$$w^{[i]} = \prod_{j=1}^{M} \frac{1}{2\pi\sigma_x\sigma_y} \exp\left(-\left[\frac{(x_{obs,j} - \mu_{x,j})^2}{2\sigma_x^2} + \frac{(y_{obs,j} - \mu_{y,j})^2}{2\sigma_y^2} \right] \right)$$

Relationship:

- Weight increases when observed data matches landmarks
- Higher error → lower weight
- σ controls sensitivity to noise

5. Normalization of Weights

$$\sum_{i=1}^{N} w^{[i]} = 1$$

Relationship:

All particle probabilities must sum to 1 to form a valid probability distribution.

6. Resampling Probability

$$P(x^{[i]}) = w^{[i]}$$

Relationship:

Particles with higher weights are more likely to be selected → improves accuracy over time.

12 System of Equations

The localization process is governed by a system of equations including:

- **State transition equations** for predicting particle positions
- **Observation equations** for transforming sensor data into map coordinates
- **Weight update equations** using multivariate Gaussian distributions
- **Normalization equations** to ensure probabilities sum to one

Together, these equations form a probabilistic framework that estimates the vehicle's position accurately.

13 What-if Analysis

- **What if GPS signal is lost?**

  The system relies more on LiDAR and IMU data, but accuracy may decrease over time.
- **What if the vehicle is kidnapped?**

  Particle filters redistribute particles globally or near GPS estimates to recover position.
- **What if sensor noise increases?**

  The system becomes less confident, leading to wider particle spread.
- **What if computational resources are limited?**

  Fewer particles are used, reducing accuracy but improving speed.
- **What if environment is featureless (e.g., tunnel)?**

  Localization becomes difficult due to lack of landmarks.

14 Tools / Technologies / Methods

- **Particle Filter (Monte Carlo Localization)** for probabilistic localization
- **Adaptive Monte Carlo Localization (AMCL)** for recovery from failures
- **Sensors**: GPS, LiDAR, IMU, cameras
- **C++ Programming** for real-time implementation
- **KD-Trees** for efficient data association
- **Distance Field Maps** for fast lookup operations
- **Sensor Fusion Techniques** combining multiple sensor outputs
- **V2X Communication** for cooperative localization between vehicles

15 Conclusion

The "Kidnapped Vehicle Problem" represents a critical fail-state in autonomous navigation, demanding robust, non-linear mathematical frameworks to ensure vehicle recovery and passenger safety. As demonstrated through this extensive literature survey, traditional linear estimators like the Kalman Filter are fundamentally incapable of maintaining the multimodal probability distributions required to solve this problem [9].

Monte Carlo Localization, driven by the Particle Filter algorithm, stands as the mathematically optimal solution. By integrating noisy GNSS bounding for initialization [14], applying Gaussian noise models to counter dead-reckoning drift during kinematic prediction [8], and utilizing highly optimized map-transformations for LiDAR data association [5], [13], the Particle Filter can successfully recover a vehicle's global pose.

The transition from theoretical mathematics to a deployable C++ architecture requires rigorous optimization. As highlighted by Lu et al. [10], utilizing contiguous memory structures, $O(1)$ Distance Field map lookups, and efficient $O(N)$ resampling wheels guarantees that the algorithm operates well within the strict sub-100ms latency limits of real-world autonomous driving. Looking to the future, the integration of Adaptive MCL with V2X networks [3] and multi-sensor fusion frameworks [4] will ultimately eradicate the risks of catastrophic localization failures, paving the way for safe, reliable Level 5 autonomy [6].

16 REFERENCES

1. M. Elbes, A. Al-Fuqaha, and M. Anan, "A Precise Indoor Localization Approach based on Particle Filter and Dynamic Exclusion Techniques," *Network Protocols and Algorithms*, vol. 5, no. 2, pp. 50-71, 2013.https://www.macrothink.org/journal/index.php/npa/article/view/3717?utm_source=chatgpt.com

2. H. Durrant-Whyte, D. Rye, and E. Nebot, "Localization of Autonomous Guided Vehicles," *Robotics Research*, pp. 613-625, 1996.https://link.springer.com/chapter/10.1007/978-1-4471-1021-7_69

3. M. Shen, D. Zhao, J. Sun, and H. Peng, "Improving Localization Accuracy in Connected Vehicle Networks Using Rao-Blackwellized Particle Filters: Theory, Simulations, and Experiments," *IEEE Transactions on Intelligent Transportation Systems*, 2017.https://arxiv.org/pdf/1702.05792

4. G. Wan, X. Yang, R. Cai, H. Li, H. Wang, and S. Song, "Robust and Precise Vehicle Localization based on Multi-sensor Fusion in Diverse City Scenes," *arXiv preprint arXiv:1711.05805*, 2020.https://arxiv.org/pdf/1711.05805

5. N. Akai, T. Hirayama, and H. Murase, "3D Monte Carlo Localization with Efficient Distance Field Representation for Automated Driving in Dynamic Environments," *2020 IEEE Intelligent Vehicles Symposium (IV)*, pp. 1588-1595, 2020.http://toriwaki.nuie.nagoya-u.ac.jp/~murase/pdf/1906-pdf.pdf

6. A. Faisal, T. Yigitcanlar, M. Kamruzzaman, and G. Currie, "Understanding autonomous vehicles: A systematic literature review on capability, impact, planning and policy," *Journal of Transport and Land Use*, vol. 12, no. 1, pp. 45-72, 2019.https://researchmgt.monash.edu/ws/portalfiles/portal/271588012/262230705.pdf

7. S. Kuutti, S. Fallah, K. Katsaros, M. Dianati, F. Mccullough, and A. Mouzakitis, "A Survey of the State-of-the-Art Localisation Techniques and Their Potentials for Autonomous Vehicle Applications," *IEEE Internet of Things Journal*, vol. 5, no. 2, pp. 829-846, 2018.https://ieeexplore.ieee.org/abstract/document/8306879

8. J. Borenstein, H.R. Everett, L. Feng, and D. Wehe, "Mobile Robot Positioning — Sensors and Techniques," *Journal of Robotic Systems*, vol. 14, no. 4, pp. 231-249, 1997.https://apps.dtic.mil/sti/tr/pdf/ADA422844.pdf

9. G. Reina, A. Vargas, K. Nagatani, and K. Yoshida, "Adaptive Kalman Filtering for GPS-based Mobile Robot Localization," *2007 IEEE International Workshop on Safety, Security and Rescue Robotics*, pp. 1-6, 2007. Adaptive_Kalman_Filtering_for_GPS_based [9].pdf

10. Y. Lu, H. Ma, E. Smart, and H. Yu, "Real-time Performance-Focused Localisation Techniques for Autonomous Vehicle: A Review," *IEEE*, 2021.https://openaccess.thecvf.com/content_cvpr_2018/papers/Dhawale_Fast_Monte-Carlo_Localization_CVPR_2018_paper.pdf

11. H. Zhou and S. Sakane, "Sensor Planning for Mobile Robot Localization—A Hierarchical Approach Using a Bayesian Network and a Particle Filter," *IEEE Transactions on Robotics*, vol. 24, no. 2, pp. 481-487, 2008.https://isprs-archives.copernicus.org/articles/XLII-1/247/2018/isprs-archives-XLII-1-247-2018.pdf

12. H. Zhou and S. Sakane, "Sensor Planning for Mobile Robot Localization—A Hierarchical Approach Using a Bayesian Network and a Particle Filter," *IEEE Transactions on Robotics*, vol. 24, no. 2, pp. 481-487, 2008.https://pure.port.ac.uk/ws/portalfiles/portal/27672337/Real_time_Performance_Focused_Localisation_Techniques_for_Autonomous_Vehicle_A_Review.pdf

13. A. Dhawale, K. S. Shankar, and N. Michael, "Fast Monte-Carlo Localization on Aerial Vehicles using Approximate Continuous Belief Representations," *CVPR*, pp. 5851-5859, 2018. www.researchgate.net

14. M. Á. de Miguel, F. García, and J. M. Armingol, "Improved LiDAR Probabilistic Localization for Autonomous Vehicles Using GNSS," *Sensors*, vol. 20, p. 3145, 2020.sensors-20-03145 [14].pdf

15. D. J. Yeong, G. Velasco-Hernandez, J. Barry, and J. Walsh, "Sensor and Sensor Fusion Technology in Autonomous Vehicles: A Review," *Sensors*, vol. 21, p. 2140, 2021.https://www.mdpi.com/1424-8220/21/6/2140